AWS Implementation Guide

Stage 2: University E-Learning Platform

Complete Step-by-Step Guide for Beginners

What You Will Build	AWS Service Used
Virtual Private Cloud (isolated network)	Amazon VPC
Virtual Machine running Node.js e-learning app	Amazon EC2 (t2.micro – Free Tier)
Video & document cloud storage	Amazon S3
Managed student records database	Amazon RDS (MySQL – Free Tier)
Firewall & access control	Security Groups + IAM

> **WARNING:** Always stay within the AWS Free Tier to avoid charges. Terminate everything after your demo.

Overview & Architecture

You will build and deploy a Node.js e-learning web application on AWS demonstrating all required cloud components. The app lets students enrol in courses, take quizzes, and view their progress — with all data stored in RDS and course files stored in S3, running on an EC2 instance inside a secure VPC.

Phase	What You Do	Time Est.
1	AWS Account Setup & IAM	15 min
2	Create VPC & Networking	20 min
3	Launch EC2 Virtual Machine	15 min
4	Install Node.js E-Learning App on EC2	20 min
5	Create S3 Bucket (Video & File Storage)	10 min
6	Create RDS Database (Student Records)	20 min
7	Connect App to RDS & S3	20 min
8	Test & Verify Everything	15 min
9	CLEANUP – Terminate All Resources	10 min

> **NOTE:** Total estimated time: ~2.5 hours. Complete Phase 9 immediately after your demo.

Phase 1 – AWS Account Setup & IAM

1.1 Create Your AWS Account

1. Go to https://aws.amazon.com and click Create an AWS Account
2. Enter your email and choose a root account password
3. Enter personal details and a valid credit card (required but not charged if you stay within Free Tier)
4. Choose the Free tier support plan
5. Sign in to the AWS Management Console at https://console.aws.amazon.com

1.2 Set Your Region

1. In the top-right corner, click the region dropdown
2. Select US East (N. Virginia) – us-east-1
3. Always check this region selector before creating any resource

1.3 Create an IAM User

1. Search for IAM in the console and open the service
2. Click Users > Add users
3. Username: elearn-admin
4. Select Provide user access to the AWS Management Console
5. Choose I want to create an IAM user
6. Attach policy: AdministratorAccess
7. Click Create user and download the CSV with login credentials
8. Sign out and sign back in using your IAM user credentials

> **TIP:** Always log in using the IAM user, never the root account.

Phase 2 – Create VPC & Networking

2.1 Create the VPC

1. Search for VPC and open the service
2. Click Your VPCs > Create VPC
3. Select VPC and more
4. Name: elearning-platform-vpc
5. IPv4 CIDR: 10.0.0.0/16
6. Availability Zones: 2, Public subnets: 2, Private subnets: 2
7. NAT Gateways: None, VPC endpoints: None
8. Click Create VPC

2.2 Create Security Groups

EC2 Security Group

1. VPC > Security Groups > Create security group
2. Name: ec2-elearn-sg | VPC: elearning-platform-vpc
3. Add inbound rules:

Type	Port	Source	Purpose
SSH	22	My IP	Connect to server
HTTP	80	0.0.0.0/0	Web traffic
Custom TCP	3000	0.0.0.0/0	Node.js app

RDS Security Group

1. Create security group: rds-elearn-sg
2. Inbound: MySQL/Aurora port 3306, source = ec2-elearn-sg (select from dropdown)
3. This ensures only EC2 can reach the database

Phase 3 – Launch EC2 Virtual Machine

3.1 Launch Instance

1. EC2 > Launch Instance
2. Name: elearning-server
3. AMI: Amazon Linux 2023 (Free tier eligible)
4. Instance type: t2.micro (Free tier)
5. Create key pair: elearn-keypair (.pem for Mac/Linux, .ppk for Windows)
6. Network: elearning-platform-vpc, public subnet, Auto-assign public IP: Enable
7. Security group: ec2-elearn-sg
8. Click Launch Instance — note the Public IPv4 address when running

> **WARNING:** Save the .pem key file safely. You cannot download it again.

3.2 Connect via SSH

Windows (PowerShell):

```
icacls elearn-keypair.pem /inheritance:r /grant:r "%USERNAME%:R"
ssh -i elearn-keypair.pem ec2-user@YOUR_PUBLIC_IP
```

Mac / Linux:

```
chmod 400 elearn-keypair.pem
ssh -i elearn-keypair.pem ec2-user@YOUR_PUBLIC_IP
```

Phase 4 – Install & Deploy the E-Learning App

4.1 Install Node.js

1. Update packages:

```
sudo dnf update -y
```

2. Install Node.js:

```
sudo dnf install -y nodejs git
```

3. Verify:

```
node --version && npm --version
```

4.2 Create the Application

1. Create project folder:

```
mkdir elearn-app && cd elearn-app
```

2. Initialise and install packages:

```
npm init -y
npm install express mysql2
```

3. Create the app file:

```
nano app.js
```

4. Paste the following code:

--- START OF app.js ---

```
const express = require('express');
const mysql   = require('mysql2');
const app     = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

const db = mysql.createConnection({
  host:     process.env.DB_HOST || 'localhost',
  user:     process.env.DB_USER || 'admin',
  password: process.env.DB_PASS || 'password',
  database: process.env.DB_NAME || 'elearndb'
});
db.connect(err => {
  if (err) console.log('DB not connected:', err.message);
  else console.log('Connected to RDS MySQL!');
});

# Home - course catalogue
app.get('/', (req, res) => {
  res.send(`<html><body style='font-
family:Arial;padding:40px;background:#f0f8ff'>
  <h1>University E-Learning Platform</h1>
  <p>AWS EC2 + Node.js + RDS MySQL</p>
  <h2>Enrol in a Course</h2>
  <form method='POST' action='/enrol'>
    Student Name: <input name='student' required /><br/><br/>
    Course: <select name='course'>
      <option>Cloud Computing</option>
      <option>Web Development</option>
      <option>Data Science</option>
      <option>Cybersecurity</option>
    </select><br/><br/>
    <button type='submit'>Enrol Now</button>
  </form>
  <br/><a href='/students'>View All Enrolments</a>
```

```
  <br/><a href='/quiz'>Take a Quiz</a>
  </body></html>`);
});

# Enrol a student
app.post('/enrol', (req, res) => {
  const { student, course } = req.body;
  db.query('INSERT INTO enrolments (student_name, course) VALUES (?,?)',
    [student, course], err => {
      if (err) return res.send('Error: ' + err.message);
      res.redirect('/students');
  });
});

# View all students
app.get('/students', (req, res) => {
  db.query('SELECT * FROM enrolments ORDER BY id DESC', (err, rows) => {
    if (err) return res.send('Error: ' + err.message);
    const html = rows.map(r =>

`<tr><td>${r.id}</td><td>${r.student_name}</td><td>${r.course}</td><td>${r.pro
gress}%</td></tr>`
    ).join('');
    res.send(`<html><body style='font-
family:Arial;padding:40px;background:#f0f8ff'>
    <h1>Student Enrolments</h1>
    <table border='1' cellpadding='8'>

<tr><th>ID</th><th>Student</th><th>Course</th><th>Progress</th></tr>${html}
    </table><br/><a href='/'>Back</a></body></html>`);
  });
});

# Simple quiz page
app.get('/quiz', (req, res) => {
  res.send(`<html><body style='font-
family:Arial;padding:40px;background:#f0f8ff'>
  <h1>Cloud Computing Quiz</h1>
  <form method='POST' action='/quiz'>
    <p>1. What does IaaS stand for?</p>
    <input type='radio' name='q1' value='correct'> Infrastructure as a
Service<br/>
    <input type='radio' name='q1' value='wrong'>  Internet as a
Service<br/><br/>
    <p>2. Which AWS service is used for object storage?</p>
    <input type='radio' name='q2' value='correct'> Amazon S3<br/>
    <input type='radio' name='q2' value='wrong'>   Amazon EC2<br/><br/>
    <button type='submit'>Submit Quiz</button>
  </form></body></html>`);
});

app.post('/quiz', (req, res) => {
  const score = Object.values(req.body).filter(v => v === 'correct').length;
  res.send(`<html><body style='font-
family:Arial;padding:40px;background:#f0f8ff'>
  <h1>Quiz Result</h1><p>You scored ${score}/2</p>
  <a href='/'>Back to Home</a></body></html>`);
});

app.listen(3000, () => console.log('E-Learning app running on port 3000'));
```

--- END OF app.js ---

5. Save: Ctrl+X → Y → Enter
6. Test the app:

```
node app.js
```

7. Open browser: http://YOUR_PUBLIC_IP:3000

> **TIP:** Take a screenshot of the e-learning homepage — evidence for your presentation!

Phase 5 – Create S3 Bucket (Video & File Storage)

1. AWS Console > S3 > Create bucket
2. Name: elearning-platform-files-YOURNAME (must be globally unique)
3. Region: us-east-1
4. Block all public access: keep checked
5. Default encryption: Enable (SSE-S3)
6. Click Create bucket
7. Upload a sample course file (PDF or image) as a test

Attach IAM Role for S3 Access

1. IAM > Roles > Create role > AWS service > EC2
2. Attach policy: AmazonS3FullAccess
3. Role name: ec2-elearn-s3-role > Create role
4. EC2 > Instances > select your instance > Actions > Security > Modify IAM role
5. Select ec2-elearn-s3-role > Update IAM role

Phase 6 – Create RDS Database (Student Records)

6.1 Create DB Subnet Group

1. RDS > Subnet groups > Create DB subnet group
2. Name: elearn-db-subnet-group
3. VPC: elearning-platform-vpc, add BOTH private subnets
4. Click Create

6.2 Launch RDS Instance

1. RDS > Create database > Standard create > MySQL 8.0 > Free tier
2. DB identifier: elearning-database
3. Master username: admin | Password: ELearnPass123!
4. Instance: db.t3.micro | Storage: 20GB gp2
5. VPC: elearning-platform-vpc | Subnet group: elearn-db-subnet-group
6. Public access: No | Security group: rds-elearn-sg
7. Initial database name: elearndb
8. Click Create database — wait 10 min for Available status
9. Copy the Endpoint address — you will need it in Phase 7

Phase 7 – Connect App to RDS & S3

7.1 Create Database Tables

1. SSH into EC2, then install MySQL client:

```
sudo dnf install -y mysql
```

2. Connect to RDS (replace YOUR_RDS_ENDPOINT):

```
mysql -h YOUR_RDS_ENDPOINT -u admin -p
```

3. Create the tables:

```
USE elearndb;
CREATE TABLE enrolments (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  student_name VARCHAR(100) NOT NULL,
  course       VARCHAR(100) NOT NULL,
  progress     INT DEFAULT 0,
  enrolled_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
SHOW TABLES;
EXIT;
```

7.2 Start App with Database Connection

1. Run the app with RDS connection (replace endpoint and password):

```
DB_HOST=YOUR_RDS_ENDPOINT DB_USER=admin DB_PASS=ELearnPass123!
DB_NAME=elearndb node app.js
```

2. Visit http://YOUR_EC2_IP:3000 — enrol a student, then check the enrolments page
3. You should see student data stored live in RDS

7.3 Keep App Running with PM2

```
sudo npm install -g pm2
DB_HOST=YOUR_RDS_ENDPOINT DB_USER=admin DB_PASS=ELearnPass123!
DB_NAME=elearndb pm2 start app.js --name elearn-app
pm2 startup && pm2 save
```

> **TIP:** Take screenshots of: the enrolment form, the students page with data, and the terminal showing Connected to RDS MySQL!

Phase 8 – Test & Verify Everything

Component	How to Verify	Expected Result
VPC	AWS Console > VPC > Your VPCs	elearning-platform-vpc with public & private subnets
EC2	EC2 > Instances	State: Running, Public IP visible
Node.js App	http://YOUR_IP:3000	E-Learning homepage loads
Course Enrolment	Submit enrolment form	Student appears in /students page
Quiz	Visit /quiz and submit	Score displayed correctly
S3	AWS Console > S3	Bucket exists with uploaded course file
RDS	AWS Console > RDS > Databases	Status: Available
DB Connection	Enrolments page shows data	Data stored in RDS MySQL
IAM Role	EC2 > Security tab	ec2-elearn-s3-role attached

Phase 9 – CLEANUP (Do This After Your Demo!)

WARNING: Delete all resources immediately after your demo to avoid AWS charges.

1. RDS: RDS > Databases > Delete (skip final snapshot, type 'delete me')
2. RDS Subnet Group: RDS > Subnet groups > Delete
3. EC2: EC2 > Instances > Terminate instance
4. S3: S3 > Empty bucket first > then Delete bucket
5. Security Groups: VPC > Security Groups > Delete ec2-elearn-sg and rds-elearn-sg
6. VPC: VPC > Your VPCs > Delete elearning-platform-vpc
7. IAM Role: IAM > Roles > Delete ec2-elearn-s3-role

TIP: Check AWS Billing > Bills to confirm $0.00 charges after cleanup.

End of AWS Implementation Guide

Cloud Computing Module | University E-Learning Platform on AWS